MATH498 Project Update: Feature Steering

Andy Holmberg and Stone Amsbaugh

Introduction 1
Proposed Work 2
Literature Review 3
Preliminary Progress 5
Next Steps 7
Works Cited 8

Introduction

Sparse autoencoders are a modern tool for interpretability on large language models that map a vector from some layer in the model to a different space. Typically, this transformed vector is specified to have much higher dimensionality than the input vector. To avoid non-trivial encodings into higher dimensions, and to enforce sparsity (which will define the utility of thai tool), an L_1 norm can penalize non-sparse encodings. Additionally, the autoencoder can transform this back into a vector of the original size, and learn to minimize the difference (L_2 norm) between the original and resulting vectors, as to preserve the information.

The utility of performing such an operation has typically been for the purposes of interpretability research. Language models must learn to represent many more concepts than they have model dimensions, so the model must represent concepts as a linear combination of multiple values. By transforming this to a sparse vector, we aim to capture certain features that may activate for more specific concepts or thoughts, and if we can identify which concepts they activate for, we unlock a sizable advantage when trying to interpret the model.

In addition, SAEs provide us with a mechanism for affecting the model behavior as well. When we have the encoded, sparse vector, we now have the ability to modify this vector in a variety of different ways. One common operation is to ‘clamp’ a feature, or overwrite its element in the vector to be a specified value, typically to a value that may be observed when the feature is active. This is what was done in the ‘Golden Gate Claude’ [1]. experiment that this project is largely inspired by. By identifying the feature in a sparse autoencoder that was associated with the Golden Gate Bridge, researchers were able to ‘clamp’ this feature to an activated value, and the model then began to answer questions, while still being perfectly intelligible, in terms of this particular landmark. Specific examples of this behavior can be found in the resources below [2].

Proposed Work

There are several potential capabilities of feature steering using sparse autoencoders. This project aims to identify where feature steering can be realistically utilized, and in these areas, how can we determine reliable methods for steering. Feature steering has already been demonstrated to have the potential to influence responses in a variety of ways. However, for them to be useful for most applications, it would be invaluable for a method to exist that can determine the operation and hyperparameters involved in steering a model in a desired way that wasn't just stabbing in the dark.

These applications center around how feature steering can affect a model's behavior. We break this into three levels in order of ambitiousness.

1. Steering Model output towards specific goals

a) **Force model to focus on a specified topic.**
It has already been clearly demonstrated that model features within the sparse autoencoder can be 'clamped' to some value in order to manipulate the model's responses. See the section on preliminary work to see how we have done this. However, more advanced intervention methods, alongside methods for feature and method hyperparameter identification would be desirable results of this investigation.

b) **Manipulate underlying model affectations.**
We explore whether we can alter certain SAE features to change certain affectations of the response. A couple questions are of interest: How is a response sensitive to certain features in the SAE feature space? Can we change the tone of the response without changing the content? Can we robustly reverse / negate the content or meaning of a response by activating the same SAE features?

2. Compare these model manipulations across different activation functions.

In addition to exploring methods of manipulating an LLM's reasoning in the sparse feature space, we plan on testing these methods across different activation functions. We use two commonly used SAEs- ReLU and TopK - and two less common SAEs that allow negative feature values. Zhu et al, [5] introduce a method called AbsTopK, and we introduce a method - one that uses the same structure as a normal SAE, but applies the shrink operator as the activation function instead of ReLU. Because we are using a non-standard activation function, we will have to train SAE's from scratch.

3. Influence model alignment or misalignment.

If there is enough time, we would like to explore methods to influence / enforce model alignment with a set of rules / ideas. This could be done by using steering methods mentioned earlier. Another method we'd like to test is finding the SAE feature representation of an idea, and projecting sparse model vectors onto the space where their inner product with said representation is greater than 0. This, in theory, could align model responses to a set of rules or ideology without the rules dominating the response. In our exploration of current literature, we have not seen anyone try this method, which could make it a novel contribution to the field if promising results are obtained.

Literature Review

Towards Monosemanticity: Decomposing Language Models With Dictionary Learning
By Bricken, Templeton, Batson, Chen, Jermyn and more from Anthropic, 2023. **[1]**

Towards Monosemanticity appears to be considered something of a landmark study on interpretability and sparse autoencoders. In particular, they are able to definitively establish (using three different methods) that the sparse autoencoder that they trained on their Claude model did indeed achieve monosemanticity, a feature representing exactly one idea.

While this initial study is primarily focused on interpretability, they additionally show the potential for feature steering. In particular, they demonstrate that activating 'base64' or 'Arabic' features can force responses to be in the selected language. This is particularly interesting, because the encoded vector is far too sparse for there to exist active features for all ideas or premises for the current text being generated. It is possible that these are unique enough instances that they do get their own active features during generation, or it is simply true that the model is compatible with additional features being injected and thus decreasing sparsity. Our goals seem to be slightly different than this, but still highly related.

Overall, they have done extensive research into identifying features and the properties of their existence, which will significantly for our own feature identification.

Steering Language Model Refusal with Sparse Autoencoders
O'Brien et al, May 2025. **[3]**

This research relates to our third objective, relating to model alignment. This originally seemed like quite a stretch, but the study shows that model safety can actually be influenced by SAEs. To backtrack, the paper uses SAEs to help the model reject nefarious requests. This was particularly done by identifying a feature for rejection, and activating it as well as a feature for Western philosophy. This is fascinating, as these aren't just tokens to output, but very broad ideas that the model seems to be able to consider, actually adopting a philosophical mindset that may wish to reject certain prompts, and also considering the possibility of rejecting a request actively.

While the study shows that the model was moderately successful in rejecting certain requests, it seems that the overall response quality declined. We had similar observations with plain clamping, so it would be interesting to see if other methods for intervention on the SAE can still influence this safeguard without the detriment to the model. In my mind, a better safeguard altogether would be a safety-focused model that is trained specifically for identifying nefarious or unethical requests, and whose only job would be to classify a prompt as acceptable or rejected before passing it to the actual language model, but that is a different conversation.

However, it is hard to say whether this is actually an alignment task or not. Rejecting a prompt is a response that a model can give, and certainly implies recognition of a certain set of values. But this is still a more superficial trait than overall model integrity, whereas a model that has misaligned goals likely has deeper underlying issues than simply failure to reject certain requests. It is nonetheless an impressive demonstration of using SAEs.

Modulating sycophancy in an RLHF model via activation steering
Panickssery, August 2023. **[4]**

Unfortunately, this paper does not use sparse autoencoders. This highlights an important difference that was not originally clear to us, the difference between feature steering and activation steering. Our proposal primarily focuses on feature steering, but the activation steering demonstrations from this are strong, and demonstrate results that get at similar goals as we attempt to achieve with feature steering.

In particular, instead of just forcing a model to talk about a certain topic, this explores influencing overall model behavior and tone of speech, which relates to our objective outlined in 1b). In this case, the 'tone' or 'affectation' is syncophancy, which relates to the agreeability, verbosity, and AI-ey tone of a model that we likely know too well. The author uses a vector related to this tone and can add and subtract it from the activations - this however doesn't transfer to SAEs as well due to their sparsity and positive-values by default.

There are some powerful results, where at one point the model was so agreeable, that when the user even suggested that some people might think that 2+2=5, the model jumped in to inform the user that 2+2 was equal to 5. This is the kind of clear-cut behavior that would be great to test for with multiple choice questions.

The primary reason that this study was interesting to us was their necessity for a method of evaluating tone in a model output. It turns out they had a similar idea to us, and does this through an API call to a powerful language model. The author shares their prompt, which we may wish to use components of, which include specifying the agent's role, establishing criteria for scoring and defining the scale, and providing examples of a couple of scores so the model can better understand what to look for. We will come back to this prompt.

AbsTopK: Rethinking Sparse Autoencoders For Bidirectional Features
Zhu, et al October 2025 **[5]**

This paper introduces a framework for including negative-valued features in an SAE. They derive a new type of SAE - AbsTopK, which takes the top K features in the SAE feature space based on absolute value. This allows for a more geometric interpretation, in addition to a higher sparsity and reconstruction accuracy.

One interesting example of the geometric interpretation that AbsTopK provides, is shown in Figure 1 of their paper - a feature that is active in the word man is active in the word woman, but takes a negative value.

We will leverage this method in (2) of proposed work. We expect the comparison of AbsTopK and the non-negative SAE activation methods yield particularly interesting results in both the evaluation of outputs and geometric interpretability of the sparse features.

Preliminary Progress

The first step in being able to poke and prod at SAEs in order to conduct naive experiments is to have a language model, an SAE, and an interface for interacting with them. It is worth stating up front that the vast majority of code for this project has been generated by Anthropic's Claude Code at our direction and review.

At this point in time, we have experimented with using 2 different language models, GPT2-Small and Gemma-3-270m-it. Larger models have been attempted, but failed to feed-forward on the CPU of our laptop in a reasonable amount of time. GPT2-Small has very high-quality pretrained SAE's, where features with scored classifications can be found on Neuronpedia, which uses an automated process for explaining SAE features described in the original paper on monosemanticity [1]. Gemma-3-270m-it is a much higher quality model, and being instruction tuned is a large advantage for our use cases. However, the quality of its feature explanations on Neuronpedia are significantly less desirable. The automatically derived explanations have very low scores, and many features go unclassified or unscored. While implementing our own process for feature identification is not just useful, but part of the goal of this project, there are fears that this indicates that the SAE has either not achieved monosemanticity, or that the features generally correspond to less actionable features. For example, the feature 'code structure keywords' is not particularly descriptive nor useful in influencing model behavior.

With these models and SAEs loaded, it was immediately clear how the naive 'clamping' of a feature (for each token, map the activations into the sparse-space using the SAE, replace the value of a certain feature with a certain value, then decode them back) can influence model output.

```
Prompt: Colorado is home to many animals, such as mountain
lions, moose, and

GPT2-Small: feral cats. Animals to look out for include bear,
coyote, bobcat, coyote,

GPT2-Small with Feature 18493 clamped to 35: sharks, and
California has a new research program in shark biology,
prompting the Atlantic Shark Foundation's Prince
```

Figure 1: Difference in responses with GPT2-Small and the same model with a feature corresponding to 'sharks' clamped at a moderate level.

However, there are three issues with this. First, we can not consistently find features that correspond to useful thoughts, let alone specific thoughts that would allow us to usefully steer model responses. Second, the clamping value of '35' is just a guess based on typical activations and experiments, and we would need more procedure in picking this value. And most importantly, simply fixing the features value seems to be a fairly crude method of intervening, and has some negative results.

Prompt: Colorado is home to many animals, such as mountain lions, moose, and

GPT2-Small with Feature 18493 clamped to 35:
(Around output token 100): "Oh, this shark is just amazing. It would go everywhere," says Shark Dr. Dennis Eryko.

(Around output token 200): Shark shark shark shark Shark Shark shark shark Shark Shark Shark Shark Shark Shark Shark

Figure 2: Demonstrating the "degradation" that occurs with naive clamping.

As can be seen in Figure 2, when using naive clamping, the model will start off only somewhat influenced by the intervention (depending on the magnitude), but eventually breaks down to low-quality responses and eventually token repetition, which we are calling degradation.

To address this, we attempted to employ different methods of intervention. It was quickly realized that scaling or incrementing the encoded activations has little effect due to their sparsity. We instead decided on a 'stochastic' method of intervention. This method takes some probability of intervention and a distribution of encoded activation values, and seems to eliminate the issue of degradation, while leading to potentially higher quality responses.

Prompt: Colorado is home to many animals, such as mountain lions, moose, and

GPT2-Small with Feature 18493 sampled from a normal distribution with probability 0.3, mean 30, and standard deviation of 20:
(Around output token 250): America is supposed to be the only place in the world that gives ocean life its due. The "law" has to be there to guide the many to the bright, honorable, undisturbed peace of shark fins.

Figure 3: Demonstrating the response quality and non-degradative behavior using the stochastic approach.

Compare the responses from Figure 2 to Figure 3, when the response is several hundred tokens deep unsupervised. It is clear that degradation does not occur. This can also lead to smoother responses, as the model is not being forced to keep the idea in its activation vector at all times, but even when it isn't there it is still reminded about it from the attention heads that grab the recent context.

Since this work, some minor code developments have enabled us to more effectively explore our originally stated goals. Using the SAELens library, we have successfully trained our own SAEs, but have not yet found success in feature identification. We also have an HTML interface for exploring feature activations, which can assist in manually exploring features.

Feature Exploration

model: GPT-2 Small (124M, raw LM) | hook: blocks.8.hook_resid_pre | 39 tokens | 258 unique feature

Show feature: feature ID Show

#	Token	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
0	<|endoftext|>	11746 (501.97)	11533 (466.14)	4078 (450.63)	13997 (427.57)	7662 (412.34)
1	Question	21479 (40.92)	7920 (15.31)	23443 (10.26)	14088 (9.74)	17604 (9.41)
2	:	4387 (28.75)	12662 (17.34)	17462 (9.96)	3813 (5.99)	21718 (5.58)
3	What	17927 (38.58)	8462 (17.13)	13992 (10.30)	19705 (9.87)	18873 (5.72)
4	is	4039 (51.14)	21184 (12.19)	18873 (6.58)	11933 (5.73)	22657 (5.22)
5	faster	23200 (33.76)	22711 (12.82)	2419 (11.65)	12007 (11.56)	12453 (7.59)
6	,	81 (16.99)	16118 (10.33)	15341 (10.28)	23838 (10.08)	6633 (6.50)
7	a	15425 (13.64)	23838 (10.09)	1574 (9.90)	24412 (8.36)	7453 (5.56)

Figure 4: Feature exploration interface

Next Steps

At this point in time, many of these steps are still rough, and involve performing experiments on hypotheses or other ideas we are wanting to investigate further.

Building upon the stochastic method for intervention, we hope to experiment with other methods for intervention that may have potential.

We also plan to define several activation functions whose domain includes negative values, such as top-bottom-k and double ended RELU. This opens up new possibilities with interventions that negate encoded activations.

However perhaps most pressing, for automating these experiments, is a method of evaluation. Until now, evaluation is done by a human who gauges the quality of a model's output. There are two automated methods we hope to test:

1. Using an LLM's API to qualitatively assess model quality and relevance to experiment goals. This would include defining criteria for success for each experiment and passing these, as well as model responses, to an API such as OpenAI's, where a high-quality model can (hopefully) consistently score results, which will be stored locally and associated with the parameters used to achieve it, in addition to the model response itself.
2. Using multiple choice questions. We already have one instruction-tuned model ready to use, so if experiments can be evaluated using multiple choice questions, this would not only be the most cost and time effective solution, but would demonstrate a clear, quantitative difference in model behavior. This means defining questions, formatting, and deciding how best to convince the model to answer in a desired format.

Additionally, training SAEs and performing experiments will likely require some text datasets that contain relevant and contrasting ideas, so we hope to search for and analyze these in the near future.

Works Cited

[1] Bricken, Templeton, Batson et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning.” Anthropic. October 2023. https://transformer-circuits.pub/2023/monosemantic-features#global-analysis

[2] Turner, Thiergart, Leech et al. “Steering GPT-2-XL by adding an activation vector.” May 2023. https://www.alignmentforum.org/posts/5spBue2z2tw4JuDCx/steering-gpt-2-xl-by-adding-an-activation-vector

[3] O’Brien et al. “Steering Language Model Refusal with Sparse Autoencoders.” November 2024. https://arxiv.org/html/2411.11296v2#S4

[4] Panickserry, “Modulating sycophancy in an RLHF model via activation steering”

[5] Zhu et al, “AbsTopK: Rethinking Sparse Autoencoders For Bidirectional Features”